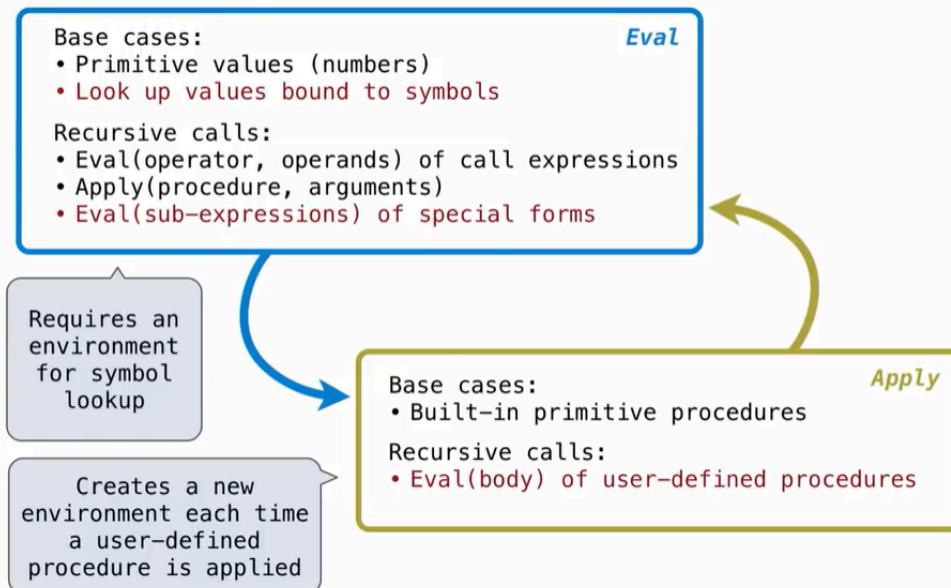


Lecture 30. Interpreters

Interpreting Scheme

The Structure of an Interpreter

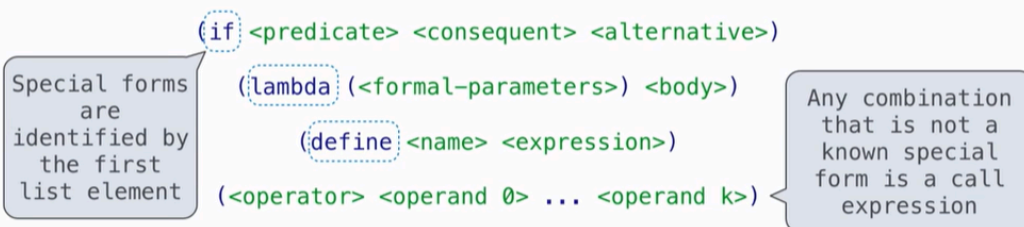


Special Forms

Scheme Evaluation

The `scheme_eval` function dispatches on expression form:

- Symbols are bound to values in the current environment.
- Self-evaluating expressions are returned.
- All other legal expressions are represented as Scheme lists, called *combinations*.



```
(define (demo s) (if (null? s) '(3) (cons (car s) (demo (cdr s)))))
```

```
(demo (list 1 2))
```

Logical Special Forms

Logical Special Forms

Logical forms may only evaluate some sub-expressions.

- **If** expression: `(if <predicate> <consequent> <alternative>)`
- **And** and **or**: `(and <e1> ... <en>)`, `(or <e1> ... <en>)`
- **Cond** expr'n: `(cond (<p1> <e1>) ... (<pn> <en>) (else <e>))`

The value of an **if** expression is the value of a sub-expression.

- Evaluate the predicate.
- Choose a sub-expression: `<consequent>` or `<alternative>`.
- Evaluate that sub-expression in place of the whole expression.

do_if_form

scheme_eval

I'm really curious about how this thing is implemented, since I have no idea how to organize all these things.

Quotation

Quotation

The **quote** special form evaluates to the quoted expression, which is **not** evaluated.

`(quote <expression>)` `(quote (+ 1 2))` evaluates to the three-element Scheme list `(+ 1 2)`

The `<expression>` itself is the value of the expression.

`'<expression>` is shorthand for `(quote <expression>)`.

`(quote (1 2))` is equivalent to `'(1 2)`

The `scheme_read` parser converts shorthand to a combination.

Lambda Expressions

Lambda Expressions

Lambda expressions evaluate to user-defined procedures.

```
(lambda (<formal-parameters>) <body>)  
  
(lambda (x) (* x x))
```

```
class LambdaProcedure:
```

```
    def __init__(self, formals, body, env):
```

```
        self.formals = formals
```

A scheme list of symbols

```
        self.body = body
```

A scheme expression

```
        self.env = env
```

A Frame instance

Frames and Environments

A frame represents an environment by having a parent frame.

Frames are Python instances with methods **lookup** and **define**.

In Project 4, Frames do not hold return values.

g: Global frame	
y	3
z	5

f1: [parent=g]	
x	2
z	4

we use the define method to bind values to the frame and add variables. then we use lookup to see the value bind to the variable in the special frame.

Define Expressions

Define Expressions

Define binds a symbol to a value in the first frame of the current environment.

```
(define <name> <expression>)
```

1. Evaluate the `<expression>`.
2. Bind `<name>` to its value in the current frame.

```
(define x (+ 1 2))
```

Procedure definition is shorthand of define with a lambda expression.

```
(define (<name> <formal parameters>) <body>)
```

```
(define <name> (lambda (<formal parameters>) <body>))
```

Applying User-Defined Procedures

To apply a user-defined procedure, create a new frame in which formal parameters are bound to argument values, whose parent is the **env** of the procedure.

Evaluate the body of the procedure in the environment that starts with this new frame.

```
(define (demo s) (if (null? s) '(3) (cons (car s) (demo (cdr s)))))
```

```
(demo (list 1 2))
```

